```c
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

#define MAX 100
int stack[MAX], top = -1;

void push(int x)
{
    if (top == MAX - 1)
    {
        printf("Stack overflow\n");
    }
    else
    {
        top++;
        stack[top] = x;
    }
}

int pop()
{
    int x;
    if (top == -1)
    {
        printf("Stack underflow\n");
        return -1;
    }
    else
    {
        x = stack[top];
        top--;
        return x;
    }
}

void display()
{
    int i;
    if (top == -1)
    {
        printf("\nStack is empty.");
    }
    else
    {
        printf("\nStack elements are:\n");
        for (i = top; i >= 0; i--)
        {
            printf("%d\n", stack[i]);
        }
    }
}

int main()
{
    int choice, x;
    while (1)
    {
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            printf("Enter the element to push: ");
            scanf("%d", &x);
            push(x);
            break;
        case 2:
            x = pop();
            if (x != -1)
                printf("Popped element: %d\n", x);
            break;
        case 3:
            display();
            break;
        case 4:
            exit(0);
        default:
            printf("Invalid choice!\n");
        }
    }
    return 0;
```

```
}
```

```c
#include <stdio.h>
#include <conio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int item) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = item;
    printf("Enqueued: %d\n", item);
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Dequeued: %d\n", queue[front++]);
}

void peek() {
    if (front == -1 || front > rear) {
        printf("Queue is Empty\n");
        return;
    }
    printf("Front Element: %d\n", queue[front]);
}

void display() {
    int i;
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    clrscr();
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    peek();
    dequeue();
    display();
    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int *L = (int*)malloc(n1 * sizeof(int));
    int *R = (int*)malloc(n2 * sizeof(int));

    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    free(L); free(R);
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int *arr;
    int n, i;
    clock_t start, end;
    double cpu_time_used;

    clrscr();
    printf("Enter number of elements (e.g., 5000): ");
    scanf("%d", &n);

    arr = (int*)malloc(n * sizeof(int));
    for (i = 0; i < n; i++) arr[i] = rand() % 10000;

    start = clock();
    mergeSort(arr, 0, n - 1);
    end = clock();

    cpu_time_used = ((double) (end - start)) / CLK_TCK;
    printf("Time taken to sort %d elements: %f seconds\n", n, cpu_time_used);

    free(arr);
    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    int j;
    for (j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    FILE *fp;
    int n, i, arr[100];
    clrscr();

    fp = fopen("data.txt", "r");
    if (fp == NULL) {
        printf("Error opening file. Create data.txt in BIN directory.\n");
        getch();
        return 1;
    }

    fscanf(fp, "%d", &n);
    for(i = 0; i < n; i++) {
        fscanf(fp, "%d", &arr[i]);
    }
    fclose(fp);

    quickSort(arr, 0, n - 1);

    printf("Sorted array from file: \n");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

int main() {
    int a[2][2], b[2][2], c[2][2];
    int p1, p2, p3, p4, p5, p6, p7, i, j;
    clrscr();

    printf("Enter 4 elements of 2x2 Matrix A:\n");
    for(i=0; i<2; i++)
        for(j=0; j<2; j++)
            scanf("%d", &a[i][j]);

    printf("Enter 4 elements of 2x2 Matrix B:\n");
    for(i=0; i<2; i++)
        for(j=0; j<2; j++)
            scanf("%d", &b[i][j]);

    p1 = a[0][0] * (b[0][1] - b[1][1]);
    p2 = (a[0][0] + a[0][1]) * b[1][1];
    p3 = (a[1][0] + a[1][1]) * b[0][0];
    p4 = a[1][1] * (b[1][0] - b[0][0]);
    p5 = (a[0][0] + a[1][1]) * (b[0][0] + b[1][1]);
    p6 = (a[0][1] - a[1][1]) * (b[1][0] + b[1][1]);
    p7 = (a[0][0] - a[1][0]) * (b[0][0] + b[0][1]);

    c[0][0] = p5 + p4 - p2 + p6;
    c[0][1] = p1 + p2;
    c[1][0] = p3 + p4;
    c[1][1] = p1 + p5 - p3 - p7;

    printf("\nProduct Matrix:\n");
    for(i=0; i<2; i++) {
        for(j=0; j<2; j++)
            printf("%d\t", c[i][j]);
        printf("\n");
    }

    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

int max(int a, int b) { return (a > b) ? a : b; }

int main() {
    int val[] = {60, 100, 120};
    int wt[] = {10, 20, 30};
    int W = 50;
    int n = 3;
    int i, w;
    int K[4][51];
    clrscr();

    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }

    printf("Maximum profit is %d\n", K[n][W]);
    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

#define MAX 30

typedef struct edge {
    int u, v, w;
} edge;

typedef struct edge_list {
    edge data[MAX];
    int n;
} edge_list;

edge_list elist;
int Graph[MAX][MAX], n;
int belongs[MAX];

void sort() {
    int i, j;
    edge temp;
    for (i = 1; i < elist.n; i++) {
        for (j = 0; j < elist.n - 1; j++) {
            if (elist.data[j].w > elist.data[j + 1].w) {
                temp = elist.data[j];
                elist.data[j] = elist.data[j + 1];
                elist.data[j + 1] = temp;
            }
        }
    }
}

void kruskal() {
    int i, j, cno1, cno2;
    int cost = 0;

    elist.n = 0;
    for (i = 1; i <= n; i++)
        for (j = 0; j < i; j++)
            if (Graph[i][j] != 0) {
                elist.data[elist.n].u = i;
                elist.data[elist.n].v = j;
                elist.data[elist.n].w = Graph[i][j];
                elist.n++;
            }

    sort();
    for (i = 0; i < n; i++) belongs[i] = i;

    printf("\nEdges in MCST:\n");
    for (i = 0; i < elist.n; i++) {
        cno1 = belongs[elist.data[i].u];
        cno2 = belongs[elist.data[i].v];
        if (cno1 != cno2) {
            printf("%d - %d (weight %d)\n", elist.data[i].u, elist.data[i].v, elist.data[i].w);
            cost += elist.data[i].w;
            for (j = 0; j < n; j++)
                if (belongs[j] == cno2) belongs[j] = cno1;
        }
    }
    printf("Total cost = %d\n", cost);
}

int main() {
    int i, j;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &Graph[i][j]);

    kruskal();
    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

#define INF 9999
#define V 5

int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    int parent[V];
    int key[V];
    int mstSet[V];
    int i, count, v, u;
    clrscr();

    for (i = 0; i < V; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (count = 0; count < V - 1; count++) {
        int min = INF, min_index;

        for (v = 0; v < V; v++)
            if (mstSet[v] == 0 && key[v] < min) {
                min = key[v];
                min_index = v;
            }

        u = min_index;
        mstSet[u] = 1;

        for (v = 0; v < V; v++)
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
    }

    printf("Edge \tWeight\n");
    for (i = 1; i < V; i++)
        printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);

    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

#define INF 9999
#define V 5

void printSolution(int dist[]) {
    int i;
    printf("Vertex \t Distance from Source\n");
    for (i = 0; i < V; i++)
        printf("%d \t %d\n", i, dist[i]);
}

int main() {
    int graph[V][V] = {
        {0, 10, 0, 5, 0},
        {10, 0, 1, 2, 0},
        {0, 1, 0, 9, 4},
        {5, 2, 9, 0, 2},
        {0, 0, 4, 2, 0}
    };
    int dist[V], sptSet[V];
    int i, count, v, u, src = 0;
    clrscr();

    for (i = 0; i < V; i++) {
        dist[i] = INF;
        sptSet[i] = 0;
    }
    dist[src] = 0;

    for (count = 0; count < V - 1; count++) {
        int min = INF, min_index;
        for (v = 0; v < V; v++)
            if (sptSet[v] == 0 && dist[v] <= min) {
                min = dist[v];
                min_index = v;
            }

        u = min_index;
        sptSet[u] = 1;

        for (v = 0; v < V; v++)
            if (!sptSet[v] && graph[u][v] && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }

    printSolution(dist);
    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

#define INF 9999

struct Edge { int u, v, w; };

int main() {
    int V = 5, E = 8, i, j;
    int dist[5];
    struct Edge edges[] = {
        {0, 1, -1}, {0, 2, 4},
        {1, 2, 3}, {1, 3, 2}, {1, 4, 2},
        {3, 2, 5}, {3, 1, 1}, {4, 3, -3}
    };
    clrscr();

    for (i = 0; i < V; i++) dist[i] = INF;
    dist[0] = 0;

    for (i = 1; i <= V - 1; i++) {
        for (j = 0; j < E; j++) {
            int u = edges[j].u;
            int v = edges[j].v;
            int w = edges[j].w;
            if (dist[u] != INF && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }

    for (i = 0; i < E; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int w = edges[i].w;
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            printf("Graph contains negative weight cycle\n");
            getch();
            return 1;
        }
    }

    printf("Vertex \t Distance from Source\n");
    for (i = 0; i < V; i++)
        printf("%d \t %d\n", i, dist[i]);

    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>
#include <math.h>

int board[20], count;

int place(int row, int column) {
    int i;
    for (i = 1; i <= row - 1; ++i) {
        if (board[i] == column || abs(board[i] - column) == abs(i - row))
            return 0;
    }
    return 1;
}

void printBoard(int n) {
    int i, j;
    printf("\n\nSolution %d:\n\n", ++count);
    for (i = 1; i <= n; ++i) {
        for (j = 1; j <= n; ++j) {
            if (board[i] == j) printf(" Q ");
            else printf(" - ");
        }
        printf("\n");
    }
}

void queen(int row, int n) {
    int column;
    for (column = 1; column <= n; ++column) {
        if (place(row, column)) {
            board[row] = column;
            if (row == n) printBoard(n);
            else queen(row + 1, n);
        }
    }
}

int main() {
    int n;
    clrscr();
    printf("Enter number of Queens: ");
    scanf("%d", &n);
    queen(1, n);
    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

int s[10], x[10], d, n, found = 0;

void sumOfSub(int sum, int k, int rem) {
    int i;
    x[k] = 1;
    if (sum + s[k] == d) {
        found = 1;
        printf("Subset: { ");
        for (i = 1; i <= k; i++)
            if (x[i] == 1) printf("%d ", s[i]);
        printf("}\n");
    } else if (sum + s[k] + s[k + 1] <= d) {
        sumOfSub(sum + s[k], k + 1, rem - s[k]);
    }

    if ((sum + rem - s[k] >= d) && (sum + s[k + 1] <= d)) {
        x[k] = 0;
        sumOfSub(sum, k + 1, rem - s[k]);
    }
}

int main() {
    int i, sum = 0;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements in increasing order:\n");
    for (i = 1; i <= n; i++) {
        scanf("%d", &s[i]);
        sum += s[i];
    }
    printf("Enter target sum: ");
    scanf("%d", &d);

    if (sum < d || s[1] > d)
        printf("No subset possible.\n");
    else
        sumOfSub(0, 1, sum);

    if (!found) printf("No subset found.\n");
    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

int a[20][20], q[20], visited[20], n, i, j, f = 0, r = -1;

void bfs(int v) {
    for (i = 0; i < n; i++) {
        if (a[v][i] && !visited[i]) {
            q[++r] = i;
            visited[i] = 1;
        }
    }
    if (f <= r) {
        bfs(q[f++]);
    }
}

int main() {
    int v;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        q[i] = 0;
        visited[i] = 0;
    }

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    printf("Enter starting vertex (0 to %d): ", n - 1);
    scanf("%d", &v);

    visited[v] = 1;
    printf("Reachable nodes: %d ", v);
    bfs(v);

    for (i = 0; i < n; i++)
        if (visited[i] && i != v)
            printf("%d ", i);

    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>

int a[20][20], visited[20], n;

void dfs(int v) {
    int i;
    visited[v] = 1;
    for (i = 0; i < n; i++) {
        if (a[v][i] == 1 && visited[i] == 0) {
            dfs(i);
        }
    }
}

int main() {
    int i, j, connected = 1;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) visited[i] = 0;

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    dfs(0);

    for (i = 0; i < n; i++) {
        if (visited[i] == 0) {
            connected = 0;
            break;
        }
    }

    if (connected)
        printf("The graph is CONNECTED.\n");
    else
        printf("The graph is NOT CONNECTED.\n");

    getch();
    return 0;
}
```

```c
#include <stdio.h>
#include <conio.h>
#include <string.h>

int main() {
    char text[100], pat[100];
    int n, m, i, j, found = 0;
    clrscr();

    printf("Enter text: ");
    gets(text);
    printf("Enter pattern: ");
    gets(pat);

    n = strlen(text);
    m = strlen(pat);

    for (i = 0; i <= n - m; i++) {
        for (j = 0; j < m; j++) {
            if (text[i + j] != pat[j])
                break;
        }
        if (j == m) {
            printf("Pattern found at index %d\n", i);
            found = 1;
        }
    }

    if (!found) {
        printf("Pattern not found in string.\n");
    }

    getch();
    return 0;
}
```